

Mise en ligne CI/CD chez la cliente (serveur Coolify)

Suivi pas-à-pas — GitHub + serveur + flux automatique. Coche chaque case au fur et à mesure.

0. L'approche en un coup d'œil

Deux environnements, une seule base de code, pilotés par la **branche Git** :

Environnement	Branche	Serveur	Reverse-proxy	Domaines
Recette (nous)	staging	Contabo	nginx mutualisé	*.gathe-finance.horus-lab.com
Production (cliente)	main	Serveur cliente Coolify	Traefik + Let's Encrypt (Coolify)	*.gathe-finance.com

Flux cible : développement → merge dans `staging` (déploie la recette automatiquement) → on teste → **Pull Request** `staging` → `main` → au merge : CI build les images : `main` (domaines cliente) → `deploy.yml` se connecte en SSH au serveur cliente → `docker compose pull + up` → le Traefik de Coolify route les sous-domaines → smoke tests.

Règle des noms dissociés : les secrets/variables du serveur cliente sont préfixés `CLIENT_*` et n'ont **aucun repli** sur les `VPS_*` (Contabo). Impossible de déployer la cliente sur le mauvais serveur par erreur : si `CLIENT_VPS_HOST` est absent, le déploiement s'ignore.

Mapping des sous-domaines cliente

Sous-domaine	Service	Géré par nous ?
app.gathe-finance.com	Vitrine (site public)	✓ oui
portail.gathe-finance.com	Espace membre	✓ oui
admin.gathe-finance.com	Back-office	✓ oui
api.gathe-finance.com	API Django + webhooks	✓ oui
cms.gathe-finance.com	Wagtail (CMS)	✓ oui
console. / traefik. / s3.	Plateforme Coolify (console, proxy, stockage)	✗ existant — ne pas toucher

1. Pré-requis (à réunir avant de commencer)

- ☐ **Accès SSH** au serveur cliente : IP/hostname, utilisateur, port.
- ☐ **Coolify opérationnel** sur le serveur (Traefik + réseau proxy déjà en place).
- ☐ **Contrôle DNS** du domaine `gathe-finance.com` (créer des enregistrements A).
- ☐ **PAT GitHub** avec le scope `read:packages` (pour tirer les images privées GHCR).
- ☐ **Secrets applicatifs** : Django `SECRET_KEY`, mot de passe Postgres, clé API Brevo, (Tara si paiement réel).
- ☐ **Clé SSH de déploiement** dédiée (paire `ed25519`, jamais ta clé perso).

2. Côté GitHub — configuration

Emplacement : Repo → Settings → Secrets and variables → Actions .

2.1 — Variables de dépôt (onglet « Variables », niveau Repository)

Pourquoi : les URLs publiques du front (Next.js) sont **figées au build** de l'image. La CI lit ces variables pour construire les images `:main` avec les bons domaines. Elles doivent être au niveau Repository (le job de build n'a pas d'Environment).

Nom de la variable	Valeur
CLIENT_SITE_DOMAIN	app.gathe-finance.com
CLIENT_PORTAL_DOMAIN	portail.gathe-finance.com
CLIENT_API_DOMAIN	api.gathe-finance.com
CLIENT_ADMIN_DOMAIN	admin.gathe-finance.com
CLIENT_CMS_DOMAIN	cms.gathe-finance.com

☐ Les 5 variables sont créées.

2.2 — Environment « production » (onglet « Environments »)

Pourquoi : on isole les **secrets SSH de la cliente** dans un Environment (permet une règle de protection / approbation avant chaque déploiement prod).

- ☐ Créer (ou ouvrir) l'Environment `production` .
- ☐ Ajouter les **Environment secrets** :

Secret	Valeur
CLIENT_VPS_HOST	IP / hostname du serveur cliente
CLIENT_VPS_USER	utilisateur SSH (ex. <code>root</code> ou <code>deploy</code>)
CLIENT_VPS_SSH_PRIVATE_KEY	clé privée ed25519 dédiée (contenu complet)
CLIENT_VPS_DEPLOY_PATH	(optionnel) <code>/opt/gathe-finance</code>

☐ (Optionnel) Protection rule : « Required reviewers » pour valider chaque déploiement prod.

À ne pas toucher : les secrets `VPS_HOST` / `VPS_USER` / `VPS_SSH_PRIVATE_KEY` existants restent tels quels — ils servent la **recette** (Contabo) via la branche `staging` .

3. Côté serveur cliente — bootstrap (une seule fois)

En SSH sur le serveur cliente.

3.1 — Relever le réseau Traefik de Coolify + le certresolver

Pourquoi : nos services doivent se rattacher au **réseau Traefik existant** de Coolify (on ne lance pas notre propre Traefik). Il faut son nom exact + le nom du « certresolver » (Let's Encrypt).

```
# Nom du réseau proxy (souvent « coolify ») :
docker network ls | grep -i coolify

# Conteneur Traefik + son certresolver (cherche --certificatesresolvers.XXX) :
docker ps --format '{{.Names}}' | grep -i proxy
docker inspect coolify-proxy 2>/dev/null | grep -i certificatesresolvers
```

☐ Noté : `PROXY_NETWORK` = _____ · `TRAEFIK_CERTRESOLVER` = _____

3.2 — Vérifier que Traefik surveille les conteneurs « externes »

Pourquoi : Coolify peut restreindre Traefik aux conteneurs qu'il gère. Si `exposedByDefault=false` avec une contrainte, nos labels doivent la satisfaire. À valider ensemble.

```
docker inspect coolify-proxy | grep -iE "exposedbydefault|constraints"
```

☐ Résultat relevé et transmis (on ajuste les labels si besoin).

3.3 — Cloner le dépôt

```
sudo mkdir -p /opt/gathe-finance && cd /opt/gathe-finance
git clone https://github.com/EdwinTchakounte/GatheFinance_project.git .
git checkout main
```

☐ Repo cloné dans `/opt/gathe-finance` sur la branche `main`.

3.4 — Créer `infra/.env.prod` (secrets — jamais commité)

Pourquoi : ce fichier contient les secrets + les domaines + le réseau/resolver Coolify. Il reste sur le serveur.

```
cp infra/.env.prod.client.example infra/.env.prod
nano infra/.env.prod # remplir TOUS les __CHANGE_ME__
```

À renseigner : `__CLIENT_DOMAIN__` → `gathe-finance.com`, `DJANGO_SECRET_KEY`, `POSTGRES_PASSWORD`, `DJANGO_SUPERUSER_PASSWORD`, `BREVO_API_KEY`, `PROXY_NETWORK` (étape 3.1), `TRAEFIK_CERTRESOLVER`, et `GATHE_IMAGE_TAG=main`.

☐ `infra/.env.prod` rempli, aucun `__CHANGE_ME__` restant.

3.5 — Connexion au registre d'images GHCR

Pourquoi : les images sont privées ; le serveur doit s'authentifier pour les tirer.

```
echo <PAT_read_packages> | docker login ghcr.io -u <user_github> --password-stdin
```

☐ Login Succeeded .

3.6 — DNS : enregistrements A

Pourquoi : Traefik/Coolify émet les certificats TLS par challenge HTTP → les domaines doivent pointer sur le serveur AVANT le 1er démarrage.

Type	Nom	Valeur
A	app	IP serveur cliente
A	portail	IP serveur cliente
A	admin	IP serveur cliente
A	api	IP serveur cliente
A	cms	IP serveur cliente

☐ 5 enregistrements créés et propagés (`dig app.gathe-finance.com` renvoie l'IP).

3.7 — Autoriser la clé de déploiement

Pourquoi : le runner GitHub se connecte en SSH avec la clé privée `CLIENT_VPS_SSH_PRIVATE_KEY` . Sa clé **publique** doit être autorisée sur le serveur.

```
ssh-copy-id -i ./gathe_deploy_client.pub <CLIENT_VPS_USER>@<IP>
# puis coller la clé PRIVÉE dans le secret GitHub CLIENT_VPS_SSH_PRIVATE_KEY
```

☐ Clé publique dans `~/.ssh/authorized_keys` , clé privée dans le secret GitHub.

4. Premier déploiement + seeds

4.1 — Déclencher le déploiement

Option A (recommandée, via CI/CD) : ouvrir une **Pull Request** `staging` → `main` et la merger. → CI build `:main` → `deploy.yml` déploie automatiquement.

Option B (1er démarrage manuel, pour valider avant d'automatiser) :

```
cd /opt/gathe-finance/infra
docker compose -f docker-compose.prod.yml -f docker-compose.coolify.yml \
  --env-file .env.prod pull
docker compose -f docker-compose.prod.yml -f docker-compose.coolify.yml \
  --env-file .env.prod up -d          # PAS de --profile traefik
```

☐ Les conteneurs démarrent ; Traefik émet les certificats (patiente 1–2 min).

4.2 — Seeds initiaux (idempotents)

```
DC="docker compose -f docker-compose.prod.yml -f docker-compose.coolify.yml --env-file .env.prod"
$DC exec backend python manage.py migrate
$DC exec backend python manage.py bootstrap_site
$DC exec backend python manage.py seed_blog
$DC exec backend python manage.py seed_email_templates --force
$DC exec backend python manage.py seed_fees
$DC exec backend python manage.py seed_rates
$DC exec backend python manage.py seed_app_settings
$DC exec backend python manage.py seed_q_schedules
```

☐ Seeds passés sans erreur.

4.3 — Compte super-administrateur

```
$DC exec backend python manage.py createsuperuser
```

☐ Superuser créé (ou via `DJANGO_SUPERUSER_*` de `.env.prod`).

5. Vérification (recette de mise en ligne)

Contrôle	Attendu	OK
<code>curl -I https://api.gathe-finance.com/healthz/</code>	200 + cert valide	☐
<code>https://app.gathe-finance.com</code>	vitrine s'affiche	☐
<code>https://portail.gathe-finance.com</code>	écran de connexion membre	☐
<code>https://admin.gathe-finance.com</code>	connexion back-office	☐
<code>https://cms.gathe-finance.com/admin/</code>	Wagtail (302 → login)	☐
Webhook Tara	<code>PUBLIC_BASE_URL=https://api.gathe-finance.com</code>	☐

6. Le cycle courant (après le bootstrap)

1. Développer sur une branche → merger dans `staging` → **recette auto (horus-lab)**
2. Tester la recette.
3. Ouvrir une **PR staging → main** → merger → **prod cliente auto**
4. En cas d'échec du healthcheck backend, le déploiement **rollback automatiquement** sur le tag précédent.

Plus jamais de manipulation serveur au quotidien : seul le **merge** déploie.

7. Dépannage rapide

Symptôme	Cause probable → action
404 Traefik sur un sous-domaine	service pas sur le réseau proxy / mauvais <code>PROXY_NETWORK</code> → vérifier <code>docker-compose.coolify.yml</code> + label <code>traefik.docker.network</code>
Certificat non émis (TLS invalide)	DNS pas encore propagé ou mauvais <code>TRAEFIK_CERTRESOLVER</code> → vérifier étape 3.1 + 3.6
<code>pull access denied</code> GHCR	PAT manquant/expiré → refaire <code>docker login ghcr.io</code> (étape 3.5)
Django « DisallowedHost / CSRF »	<code>DJANGO_ALLOWED_HOSTS</code> incomplet dans <code>.env.prod</code>
Déploiement GitHub « ignoré »	<code>CLIENT_VPS_HOST</code> non renseigné → étape 2.2
Front affiche les mauvaises URLs	images <code>:main</code> buildées avant de poser les variables <code>CLIENT_*</code> → re-lancer la CI